

X3 ATOMIC STAR · X3 CHAIN

x3-lang

A practical guide to the cross-VM intent language,
its compiler,
its virtual machine, and what it can actually do
today.

Scope: the x3-lang workspace at `x3-lang/` in the X3 Chain monorepo

Grounded in: source code, grammar spec, opcode tables, and passing tests — not roadmap prose

Verified: every code example in this book was compiled with the real `x3c` toolchain

Prepared: September 2026

Contents

1	What Is x3-lang?	3
2	Core Concepts	5
3	Two Languages in One Workspace	8
4	Syntax Reference	9
5	Worked Examples	12
	5.1 A cross-chain swap	
	5.2 A hashlock atomic swap (HTLC)	
	5.3 Timeout and refund	
	5.4 Cross-VM staking	
	5.5 Bounded venue fallback	
	5.6 Flash-loan arbitrage (Trading Core v1)	
6	The Compiler Pipeline	18
7	The Virtual Machine	20
8	The x3c Toolchain	22
9	Proof by Test	26
10	Current Status & Honest Limitations	27
A	Appendix A — Full Opcode Table	29
B	Appendix B — Quick Reference Card	33

CHAPTER 1

What Is x3-lang?

x3-lang is a domain-specific language, compiler, and virtual machine for expressing *cross-chain intents* — statements like “move this asset from chain A to chain B, subject to these guards, with this timeout and refund behavior” — that compile down to a custom bytecode and run on a purpose-built register machine called the X3VM. It is the language layer for X3 Chain's multi-VM architecture (X3VM + EVM + SVM), and its job is to make statements about money that touch more than one blockchain *atomic*: either every leg of a route settles, or none of them do.

The central engineering bet behind x3-lang is that cross-chain operations are dangerous precisely because they are usually assembled by hand out of several independent transactions, coordinated by off-chain scripts, with no single point that can guarantee all-or-nothing behavior. A bridge transfer that succeeds on the source chain but fails on the destination chain is not a bug in any one transaction — it is a coordination failure between transactions that no single chain's execution model can see. x3-lang's answer is to make the coordination itself a first-class, compiled, verified program: the same artifact that describes *what* should happen also carries the guards, timeouts, and rollback rules for what happens when it doesn't, and the VM enforces the atomic commit-or-rollback contract directly with dedicated opcodes (`ATOMIC_BEGIN` / `ATOMIC_END` / `ATOMIC_ROLLBACK`).

VERIFIED CLAIM

This is not aspirational. `vm/tests/property_tests.rs` fuzzes register state through `ATOMIC_BEGIN → ADD → ATOMIC_ROLLBACK` with `proptest` and asserts, for every generated input, that registers land back exactly where they started. A second test, `vm/tests/test_e2e_examples.rs::e2e_atomic_rollback_on_bridge_failure`, compiles a real `.x3` file, wires a bridge backend that deliberately fails mid-route, runs it on the real VM, and asserts the rollback path fires end to end — with an `unreachable!()` guard proving the proof-verification and receipt-persistence steps are never reached on the failure path. Chapter 9 goes through this in detail.

Why a new language, rather than a library?

Three properties are hard to bolt onto an existing general-purpose language after the fact, and x3-lang bakes them into the grammar instead:

- **Atomicity as syntax, not convention.** An `atomic swap` or `atomic trade` block is a single compiled unit with one entry and a VM-enforced rollback boundary — a programmer cannot forget to handle the failure path, because there is no way to express the happy path without also compiling the guards.
- **Bounded, auditable substitution.** Where a route might need a fallback (a different DEX, a different bridge), x3-lang requires every fallback to be named and cost-bounded at compile time (`fallback { replace with curve; replace with balancer min_output 1 }`). There is deliberately no syntax for “try anything that works” — see Chapter 5.5.

- **Receipts as a language-level concept, not an afterthought.** Settlement produces a signed, independently verifiable receipt (`emit receipt` , `x3c receipt verify`) rather than leaving proof-of-execution to whatever the caller happens to log.

Two audiences, one workspace

The `x3-lang/` directory actually contains two coexisting implementations, and the repository is explicit about which one is authoritative for what:

Surface	Files	Authoritative for
Python MVP	<code>cli.py</code> , <code>registry.py</code> , <code>typechecker.py</code> , <code>planner.py</code> , <code>simulator.py</code> , <code>emitter/</code> , <code>runner.py</code>	The shipping intent DSL prototype and MVP pipeline
Rust production workspace	<code>Cargo.toml</code> , <code>compiler/</code> , <code>vm/</code> , <code>crates/x3-ast</code> , <code>crates/x3-common</code> , <code>crates/x3-lexer</code> , <code>crates/x3-tools</code>	The compiler, IR, opcode set, and VM bytecode semantics

This book is about the Rust production workspace — the compiler, the IR, the VM, and the `x3c` CLI — because that is what the repository itself designates as authoritative for language semantics. Every code sample in this book was compiled against that toolchain.

CHAPTER 2

Core Concepts

Intents

An **intent** is the basic unit of cross-chain desire: a named block that declares a source asset and amount, a destination asset and receiver, a **route** describing how to get from one to the other, a list of **require** guards that must hold before the route is allowed to execute, and a **timeout / refund** pair describing what happens if the route does not settle in time.

```
intent simple_swap {
  from arbitrum.USDC amount 500
  to solana.SOL receiver wallet

  route {
    bridge X3 arbitrum.USDC -> solana.SOL receiver wallet
  }

  require nonce unused simple_swap_001
  require finality.arbitrum >= 32
  require slippage <= 50

  timeout 180s refund arbitrum.USDC to sender
  on_fail rollback
}
```

The shape every intent shares: **from / to**, a **route**, a list of **require** guards, and a **timeout/refund** escape hatch.

Guards (**require**)

Guards are preconditions the compiler and VM both understand. They are not comments about intent — they lower to real opcodes (**REQUIRE** , **NONCE_UNUSED** , **PROOF_REQUIRED** , **ROUTE_SCORE** , and others) that the VM evaluates before it will let a route proceed. Typical guard kinds seen across the example programs:

Guard	Meaning
require nonce unused <id>	Replay protection — this intent id has not executed before
require finality.<chain> >= N	Source/destination chain must have at least N confirmations
require finality <chain> == finalized	Chain-specific finality predicate (space-separated form)
require slippage <= N	Maximum tolerated slippage, in basis points

<code>require route_score >= N</code>	Minimum score from the route-scoring pass (Chapter 6)
<code>require relayer_quorum >= N</code>	Minimum number of attesting relayers (cross-checked against a declared <code>relayers {}</code> set)
<code>require net_profit >= N <ASSET></code>	Trading Core: the trade must clear a minimum profit before it is allowed to commit
<code>require all_debts_repaid</code>	Trading Core: every flash-borrowed debt must be closed before commit

TWO SPELLINGS, ONE GUARD

The compiler accepts both `require finality.arbitrum >= 32` (dotted subject) and `require finality Ethereum >= 64` (space-separated subject) — both are real, separate branches in `parser.rs::parse_require_guard`. A stale test comment elsewhere in the repo claims the space-separated form isn't implemented; it is. This is a small but concrete example of why this book treats compiled output, not comments or older docs, as ground truth — exactly the rule this repository's own contribution guidelines state.

Atomicity

Atomicity in x3-lang means one of two outcomes, never a third: every leg of a route commits, or every leg that already ran is rolled back to its pre-execution state. This is implemented at the VM level, not just as a compiler convention:

- `ATOMIC_BEGIN` snapshots VM state (registers, in-flight asset operations, bridge receipts).
- `ATOMIC_END` commits that state if every guard passed.
- `ATOMIC_ROLLBACK` restores the snapshot exactly, discarding any partial effects.

Trading Core programs use a parallel but equivalent set of opcodes (`TRADING_BEGIN` ... `TRADING_COMMIT` / `TRADING_ABORT`) built around borrow/swap/repay instead of lock/mint/burn, but the underlying contract — full commit or full rollback, nothing in between — is the same.

Cross-VM execution

A single route can name assets that live on an EVM chain, an SVM chain, and X3's own bridge primitive in the same block. The VM dispatches chain-specific work through adapters: `EMIT` for EVM-side calls, `CALL_HOST` for SVM-side calls, and `BRIDGE` together with a pluggable bridge backend (light-client or RPC-finality verifiers for both EVM and SVM families) for moving assets between them. The language does not need chain-specific syntax for this — the same `route { bridge X3 ... }` or `swap ... via uniswap_v3` declarations compile to the same opcode family regardless of which chains the named assets live on.

Receipts

Settlement produces a receipt: a deterministic, hashable record of what happened, which is Ed25519-signed and independently verifiable (`build_receipt` → `sign_receipt` → `verify_receipt_trusted` , exposed to users as `x3c receipt verify`). A receipt is not a log line — it is the thing a counterparty or auditor is meant to check instead of trusting the executor's word.

CHAPTER 3

Two Languages in One Workspace

Reading the Rust compiler's parser reveals that x3-lang is really two languages sharing one toolchain: a small, general-purpose imperative core, and a much larger, more mature declarative DSL for intents and trades. Almost everything in Chapters 2 and 5 belongs to the second layer. Understanding both — and which one you are looking at — avoids a common confusion when reading real `.x3` files side by side.

Layer 1 — the imperative core

Documented in `docs/openspec/x3-language-grammar.md`, this layer looks like a small C-family language: functions, `let` bindings, `if / while / for / loop`, an `atomic(expr) { }` block, and an `emit` statement. Its full statement grammar is reproduced in Chapter 4. It compiles and runs — a minimal program such as `fn simple_fn() { let x = 42; }` genuinely lowers and executes — but in practice it is a thin substrate underneath the declarative layer rather than how real X3 programs are written.

Layer 2 — the declarative intent / trading DSL

This is the layer every realistic example in this book uses: `finality_policy`, `risk_policy`, `intent`, `route { swap / bridge / fallback }`, `require`, `timeout ... refund ...`, `on_fail`, `proofs required { }`, `atomic swap A -> B { ... }` (hashlock/HTLC style), and the Trading Core v1 vocabulary: `asset NAME = vm.chain.address { decimals: N }`, `risk policy Name { ... }`, and `atomic trade Name using Policy { borrow ... ; swap ... via venue min_out ... ; repay ...; require ... ; emit receipt }`. None of this is in the openspec grammar file — it is best learned from the parser's own function names (`parse_finality_policy`, `parse_intent`, `parse_atomic_swap`, `parse_atomic_trade`, and roughly forty siblings) and from the worked examples in Chapter 5, which is exactly how this book was written.

A TRAP TO AVOID

The `x3-lang/examples/` directory also contains a handful of files (`arb.x3`, `flash.x3`, `jit_lp.x3`, `mev_smooth.x3`, `x3_coin_layer.x3`) written in a third, Solidity-flavored dialect — `contract { }`, `u256`, `event Foo(...)`, `@swarm` / `@atomic` decorators. These do not parse with the real toolchain (`x3c check` fails with “*expected top-level item*” on every one of them). They are left over from an earlier design iteration and are not x3-lang as it exists today. Chapter 10 has the full list of what compiles and what doesn't.

CHAPTER 4

Syntax Reference

4.1 Imperative core grammar

Verbatim from `docs/openspec/x3-language-grammar.md` (86 lines total):

```

program      ::= { item } EOF ;
item         ::= function_decl | global_let | agent_decl | import_decl | const_decl ;
function_decl ::= "fn" identifier "(" [ param_list ] ")" [ "->" type ] block ;
global_let   ::= "let" identifier [ ":" type ] "=" expression ";" ;
block        ::= "{" { stmt } "}" ;

stmt         ::= let_stmt | expr_stmt | return_stmt | atomic_block | if_stmt
              | while_stmt | loop_stmt | for_stmt | emit_stmt ;

let_stmt     ::= "let" "mut"? identifier [ ":" type ] "=" expression ";" ;
return_stmt  ::= "return" expression? ";" ;
expr_stmt    ::= expression ";" ;
atomic_block ::= "atomic" "(" expression ")" block | "atomic" block ;
if_stmt      ::= "if" "(" expression ")" block [ "else" ( block | if_stmt ) ] ;
while_stmt   ::= "while" "(" expression ")" block ;
loop_stmt    ::= "loop" block ;
for_stmt     ::= "for" "(" ( let_stmt | expr_stmt | ";" ) expression? ";" expression? ")" block ;
emit_stmt    ::= "emit" identifier "(" [ expression { "," expression } ] ")" ";" ;

```

Expression precedence

Standard Pratt hierarchy, highest binding first:

```

postfix (call / index / field) > unary (! - +) > * / % > + - > << >> > < > <=
>= > == != > & > ^ > | > && > || > ?: > = > ,

```

4.2 Declarative DSL — building blocks

These forms are not in the openspec file but are the real, load-bearing surface, confirmed against `parser.rs` and every compiling example in `examples/`.

Finality and risk policy declarations

```
finality_policy strict {
  chain arbitrum
  requirement finalized
  blocks 32
}

risk_policy {
  min_route_score 90
}
```

Intents and routes

```
intent <name> {
  from <chain>.<ASSET> amount <N> [receiver <addr>]
  to   <chain>.<ASSET> receiver <addr>

  route {
    bridge <bridge> <chain>.<ASSET> -> <chain>.<ASSET> receiver <addr>
    | swap <venue> <chain>.<ASSET> -> <chain>.<ASSET> amount <N> min_output <N>

    fallback {
      replace with <venue>
      require slippage <= <N>
      require profit >= <N>
    }
  }

  require <guard> ...
  timeout <duration> refund <chain>.<ASSET> to sender
  on_fail rollback
}

proofs required {
  source_lock_proof
  destination_fill_proof
}
```

Hashlock atomic swaps

```
atomic swap <chainA>.<ASSET> -> <chainB>.<ASSET> {
  amount <N>
  receiver <addr>
  hashlock sha256(secret)
```

```

    timeout source <duration>
    timeout destination <duration>
    require finality.<chain> >= <N>
    require relayer_quorum >= <N>
}

```

Trading Core v1 — typed assets, risk policy, atomic trades

```

asset <NAME> = <vm>.<chain>.<address> {
    decimals: <N>
}

risk policy <Name> {
    max_slippage: <N> bps
    max_gas: <N> ETH
    max_flash_fee: <N> bps
    deadline: <N> blocks
    require_private_submission: true
}

atomic trade <Name> using <Policy> {
    borrow <N> <ASSET> from <venue> as debt

    let <x> = swap debt.amount <ASSET> -> <ASSET>
        via <venue>
        min_out <N> <ASSET>

    repay debt

    require net_profit >= <N> <ASSET>
    require all_debts_repaid
    emit receipt
}

```

DESIGN NOTE: NO DYNAMIC SUBSTITUTION

Notice there is no "try any DEX with the best price" construct anywhere in this grammar. Every venue, every fallback, and every bound (`min_out` , `max_slippage`) is named and checked at compile time. The example programs' own comments describe this directly as the syntax cost of ruling out arbitrary dynamic contract substitution — a deliberate safety trade-off, not a missing feature.

CHAPTER 5

Worked Examples

Every example in this chapter is reproduced verbatim from `x3-lang/examples/` and was compiled with `x3c` `check` against the real toolchain while writing this book.

5.1 A cross-chain swap

COMPILES · 19 ops

examples/simple_swap.x3

```

finality_policy strict {
  chain arbitrum
  requirement finalized
  blocks 32
}

finality_policy strict {
  chain solana
  requirement finalized
  blocks 32
}

risk_policy {
  min_route_score 90
}

intent simple_swap {
  from arbitrum.USDC amount 500
  to solana.SOL receiver wallet

  route {
    bridge X3 arbitrum.USDC -> solana.SOL receiver wallet
  }

  require nonce unused simple_swap_001
  require finality.arbitrum >= 32
  require finality.solana >= 32
  require slippage <= 50
  require route_score >= 90

  timeout 180s refund arbitrum.USDC to sender
  on_fail rollback
}

proofs required {

```

```

    source_lock_proof
    destination_fill_proof
}

```

500 USDC moves from Arbitrum to SOL on Solana through the X3 bridge. Both chains declare a 32-block finality requirement; the route must score at least 90 and keep slippage under 50 bps; the nonce guard stops the exact same intent from being replayed. If the route hasn't settled within 180 seconds, the sender is refunded on Arbitrum and the whole intent rolls back. The `proofs required` block is not decoration — it tells the verifier which cryptographic proofs (a lock proof on the source, a fill proof on the destination) it must demand before it will accept the bridge leg as settled.

5.2 A hashlock atomic swap (HTLC)

COMPILES · 12 ops

examples/atomic_swap.x3

```

finality_policy strict {
    chain eth
    requirement finalized
    blocks 12
}

finality_policy strict {
    chain sol
    requirement finalized
}

relayers {
    quorum_numerator 3
    quorum_denominator 5
    relayers [relayer_a, relayer_b, relayer_c, relayer_d, relayer_e]
}

atomic swap eth.USDC -> sol.SOL {
    amount 500
    receiver sol.wallet.owner
    hashlock sha256(secret)
    timeout source 40m
    timeout destination 20m
    require finality.eth >= 12
    require finality.sol == finalized
    require relayer_quorum >= 3
}

```

A classic hash-time-locked-contract swap — 500 USDC on Ethereum for SOL on Solana, unlocked by revealing the preimage of `sha256(secret)`. Source and destination legs get independent timeouts (40 and 20 minutes),

Ethereum needs 12-block finality, Solana needs to be in a finalized state, and the swap additionally requires attestation from at least 3 of the 5 relayers named in the `relayers {}` block above it — the compiler cross-checks the `relayer quorum` guard against that declared set.

5.3 Timeout and refund

COMPILES · 14 ops

examples/timeout refund.x3

[illegible]

100 USDC bridges Ethereum → Solana behind a strict 64-block finality requirement. If it hasn't settled within 45 seconds, the sender gets refunded on Ethereum. This file is a good place to notice the guard's alternate spelling: `require finality Ethereum >= 64` (space-separated) instead of the dotted `require finality.arbitrum >= 32` seen in 5.1 — both are accepted by the real parser (see the note in Chapter 2).

5.4 Cross-VM staking

COMPILES · 18 ops

examples/staking intent.x3

```
finality_policy strict { chain ethereum requirement finalized blocks 32 }
finality_policy strict { chain solana requirement finalized blocks 32 }

risk_policy {
    min route score 85
}
```

```

}

intent cross_vm_stake {
  from ethereum.ETH amount 10
  to solana.stakedSOL receiver vault

  route {
    bridge X3 ethereum.ETH -> solana.stakedSOL receiver vault
  }

  require nonce unused cross_vm_stake_001
  require finality.ethereum >= 32
  require finality.solana >= 32
  require route_score >= 85

  timeout 300s refund ethereum.ETH to sender
  on_fail rollback
}

proofs required {
  source_lock_proof
  destination_fill_proof
}

```

The same intent template as 5.1, applied to a staking endpoint instead of a plain swap: 10 ETH goes in, a staked-SOL position on Solana comes out, gated by both chains' finality, a route-score floor, and a 300-second timeout/refund/rollback safety net. Notice how little changes between a "swap" intent and a "stake" intent — the destination is just a different asset with a different settlement meaning, and the guard machinery around it is identical.

5.5 Bounded venue fallback

COMPILES · 14 ops

examples/route fallback.x3

[illegible]

```

        replace with balancer min_output 1
        require slippage <= 7
        require profit >= 0
    }
}

require nonce unused rebalance_with_fallback_1
require slippage <= 50
require route_score >= 90

timeout 120s refund ethereum.USDC to sender
on_fail rollback
}

```

Swaps 5,000 USDC → ETH on Uniswap, but with a pre-approved, compiler-checked fallback list: if the primary venue can't clear it, try Curve, then Balancer — each with its own slippage and profit floor. This is the example that most clearly shows the "no arbitrary substitution" design principle from Chapter 4: the set of venues this intent could ever route through is fixed at compile time, not discovered at runtime.

5.6 Flash-loan arbitrage — Trading Core v1

COMPILES · 9 ops

examples/trading_core_v1.x3

```

asset USDC = evm.ethereum.0xA0b8 {
    decimals: 6
}

asset WETH = evm.ethereum.0xC02a {
    decimals: 18
}

asset ETH = evm.ethereum.0x0000000000000000000000000000000000000000000000000000000000000000 {
    decimals: 18
}

risk policy MainnetArb {
    max_slippage: 30 bps
    max_gas: 0.02 ETH
    max_flash_fee: 10 bps
    deadline: 2 blocks
    require_private_submission: true
}

atomic trade CrossDexArb using MainnetArb {
    borrow 1_000_000 USDC from aave_v3 as debt
}

```

```

    let weth = swap debt.amount USDC -> WETH
      via uniswap_v3
      min_out 410 WETH

    let returned = swap weth WETH -> USDC
      via sushiswap
      min_out 1_002_000 USDC

    repay debt

    require net_profit >= 1_000 USDC
    require all_debts_repaid
    emit receipt
  }

```

This is the most complete example in the language: three chain-qualified typed assets with explicit decimals, a named risk policy (30 bps max slippage, 0.02 ETH max gas, 10 bps max flash-loan fee, a 2-block deadline, and mandatory private-mempool submission to defend against frontrunning), and an `atomic trade` that flash-borrows 1,000,000 USDC from Aave v3, routes it through Uniswap v3 into WETH and back through SushiSwap into USDC — each leg with its own minimum-output floor — repays the flash loan, and only then asserts net profit and full debt closure before emitting a signed receipt. If any `require` fails, nothing commits: not the borrow, not either swap, not the repay.

WHAT THIS EXAMPLE DOES NOT PROVE

Compiling and even executing this program against the toolchain's fixture host proves the compile → execute → receipt pipeline is wired correctly end to end. It does not prove Aave v3, Uniswap v3, or SushiSwap integration is live, and it does not simulate real market conditions — a program that needs genuine price movement to clear its own `min_out` / `net_profit` guards can still legitimately fail against a real venue. See Chapter 10.

CHAPTER 6

The Compiler Pipeline

Read directly from `compiler/src/lib.rs`, which documents itself as reconciling three verification layers "so no single check is duplicated." The real pipeline, in order:

1. **Parse** — `parser::parse_source` turns source text into an AST (`Program`).
2. **AST-level checks** — `ast_level_errors` runs a large family of structural verifiers: atomic-swap and atomic-choice declarations, route fallbacks, privacy/venue/strategy/objective/parallel declarations, solver-bond and relayer-quorum checks, proof-completeness, route-score bounds, finality guards, risk-policy bound-guards, VM-support checks, invariant guards, bridge-liquidity checks, guard-kind-checkability, and domain-specific verifiers for hedging, liquidation, rebalancing, netting, and arbitrage patterns — plus a numeric policy check on integer literals and coercions.
3. **Trading-specific semantic analysis** — for `atomic trade` programs only:
`trading_semantic::analyze_trading` and `trading_verify::verify_trading_program` check typed assets, risk policy bounds, and debt structure.
4. **Lowering** — `lowering::lower_program_with_mode` turns the AST into **X3IR**, a flat, typed intermediate representation (`Operation` , `Condition` , `FailureAction` , `RequireKind`).
5. **IR-level validation** — `ir_level_errors` and `verify::verify_ir` check structural invariants: atomic-block scoping balances, non-zero amounts and iteration counts, and empty-field safety.
6. **Whole-program semantic safety** — `semantic::verify_canonical_supply` and `semantic::verify_collect` enforce chain/adaptor/asset allow-lists, mandatory refund paths, explicit finality and proof obligations, route scores, invariant rules, and compile-mode gating (Dev / Testnet / Mainnet). This stage also collects non-fatal warnings, including profitability estimates the compiler is explicit are *"a static estimate, not a guarantee of runtime profitability."*
7. **Emission** — `emitter::emit_x3ir` turns IR into a versioned, 4-byte-aligned bytecode stream (byte 0 is the version marker).
8. **Bytecode verification** — `verify_bytecode` sanity-checks the emitted bytes (non-empty, correct version, correct alignment) before anything is allowed to run.
9. **Execution** — the VM re-verifies bytecode on its own before calling the internal fetch-decode-execute loop. A doc comment in the VM notes this path used to be reachable *unverified* through a public executor function, and was deliberately tightened to "exactly one public way in."
10. **Commit or rollback** — `ATOMIC_BEGIN/END/ROLLBACK` (or, for trades, `TRADING_BEGIN` ... `TRADING_COMMIT` / `TRADING_ABORT`) enforce the all-or-nothing contract.
11. **Settlement** — receipts are built, signed, and persisted (`build_receipt` , `sign_receipt` , `persist_receipt`).
12. **Independent verification** — receipts can be re-checked later without re-executing anything, via `verify_receipt_trusted` / `verify_receipt_economics` , exposed as `x3c receipt verify` .

CROSS-CHECK AGAINST THIS PROJECT'S OWN ATOMIC EXECUTION MODEL

This repository's contribution rules describe a canonical 12-stage atomic execution model: parse → semantic check → lower to typed IR → validate IR → preflight legs → reserve/lock assets → execute route → verify receipts/proofs → commit legs → rollback failed legs → settle final state → assert invariants. Every one of those stages has a real, working counterpart in the pipeline above — this is a genuine implementation match, not aspirational documentation. The granularity differs (the real pipeline is built from more, smaller, differently-named passes, and "preflight" is folded into the IR/semantic validation stages plus separate pre-execution tooling like `x3c estimate` and `x3c graph` rather than being its own VM phase), but every named guarantee — semantic checking, typed IR, IR validation, asset locking, route execution, receipt verification, commit, rollback, settlement, and invariant assertion — is real and test-covered (Chapter 9).

A pipeline that fixed itself

Worth noting as an example of the codebase's own hardening discipline: doc comments around the pipeline's entry-point function explicitly record that this unified funnel was *added* because the checks used to not be uniformly wired — some entry points into the compiler previously ran no AST-level or IR-level checks at all. The current single-funnel design is the fix for a real, previously-existing gap, not a feature that was always there.

CHAPTER 7

The Virtual Machine

The X3VM is a register machine with gas metering. Programs are streams of opcodes (one byte each, defined in `spec/opcodes.rs`, the file both the compiler and VM crates `include!` directly — making it the actual compiled source of truth, ahead of the separate `spec/opcodes.yaml` reference file, which is missing several newer opcodes). The full merged table is in Appendix A; the categories are:

Category	Examples	Purpose
Arithmetic / misc	ADD SUB NOP HALT	Register arithmetic, program bookends
Metadata	META_NONCE META_CHAIN_ID	Transaction nonce and chain identity
Asset ops	LOCK MINT BURN RELEASE SWAP	Reserve, mint, burn, release, and swap assets
Bridge	BRIDGE	Cross-chain transfer with proof verification
Control flow	IF LOOP CALL RET	Branching, bounded loops, subroutines
Guards	REQUIRE ON_FAIL ON_TIMEOUT	Preconditions and failure/deadline handlers
Atomic	ATOMIC_BEGIN ATOMIC_END ATOMIC_ROLLBACK	Snapshot / commit / restore VM state
External calls	EMIT CALL_HOST	EVM-side and SVM-side calls via bridge adapter
Capability dispatch	GPU_DISPATCH SIMULATE PROOF_VERIFY MEMPOOL_SCAN ...	28 host-gated capability opcodes
B-52 feature lock	ROUTE_SCORE RELAYER_ATTEST RISK_SCORE INVARIANT_CHECK ...	Route/risk scoring, relayer attestation, mode gating
Trading core	TRADING_BEGIN TRADING_OPEN_DEBT TRADING_EXECUTE_SWAP TRADING_COMMIT ...	Borrow / swap / repay / assert / commit lifecycle

ON THE README'S "26 OPCODES" CLAIM

The README states "all 26 opcodes execute," referring to a simplified summary table that lumps all 28 capability-dispatch opcodes into a single row. It is not a literal count of distinct opcode mnemonics — `opcodes.rs` defines on the order of 70–90 named, non-reserved opcodes once every category is counted individually (Appendix A). Both statements are honest, but they are answering different questions; this book uses the full table.

Gas metering

Every instruction has a base cost plus, for payload-carrying opcodes, an operand-dependent surcharge (roughly `payload_len / 32` for `EMIT`, `CALL_HOST`, `BRIDGE`, and the capability opcodes). Representative base costs: `ADD / SUB` = 1, `META_*` = 5, `LOCK / MINT` = 1, `BRIDGE` = 100, `IF / LOOP` = 2, `REQUIRE` = 10, `ATOMIC_BEGIN / ATOMIC_END` = 250, `GPU_DISPATCH` = 500, `PROOF_VERIFY` = 500, `HALT` = 0. A flat "code-deposit" cost equal to bytecode length is deducted up front. Gas never refunds and never goes negative.

Atomic commit and rollback, mechanically

`ATOMIC_BEGIN` takes a snapshot of registers, in-flight asset operations, and bridge receipts. Every opcode inside the atomic region executes against live state as normal. `ATOMIC_END` discards the snapshot and keeps the live state (commit). `ATOMIC_ROLLBACK` discards the live state and restores the snapshot exactly — including undoing any locks, mints, or partial bridge transfers that ran inside the region. A `REQUIRE` that fails inside an atomic region traps to the nearest registered `ON_FAIL` handler, which typically routes to `ATOMIC_ROLLBACK`. Trading Core uses the same pattern with `TRADING_ABORT` in place of `ATOMIC_ROLLBACK`.

Trading Core execution sequence

```
TRADING_BEGIN
→ TRADING_OPEN_DEBT (borrow)
→ TRADING_EXECUTE_SWAP × N (each named venue leg)
→ TRADING_CLOSE_DEBT (repay)
→ TRADING_ASSERT_MIN_PROFIT / TRADING_ASSERT_ALL_DEBTS / TRADING_ASSERT_INVARIANT
→ TRADING_EMIT_RECEIPT
→ TRADING_COMMIT (or TRADING_ABORT on any failed assertion)
```

This executes against a pluggable `TradingHost / TradingVm` trait — real venue adapters in a production deployment, or an explicit, clearly-labeled fixture host in tests and in `x3c receipt execute` (Chapter 8, Chapter 10).

Bridge backends

`init_production_backend()` in `vm/src/bridge.rs` wires four verifier families, each configured entirely through environment variables: `evm-light-client`, `svm-light-client`, `evm-rpc`, and `svm-rpc`. The code's own documentation states it never silently falls back to a dry-run mode if a verifier isn't configured — it fails rather than pretending to have checked something it didn't.

CHAPTER 8

The x3c Toolchain

All commands below are real subcommands of the Rust CLI, taken directly from the argument parser in [crates/x3c-tools/src/bin/x3c.rs](#). Invoke as:

```
cargo run -p x3c-tools --bin x3c -- <subcommand> [args] [--mode dev|testnet|mainnet] [--deny-warning]
```

Command	What it does
parse <file>	Parse to AST and dump JSON
check <file>	Lower to X3IR and run the full semantic verifier
lower <file> -o <out>	Lower AST to X3IR JSON
build <file> -o <out> [--provenance p]	Compile to bytecode; optionally emit a provenance document (source hash, artifact hash, compiler version, commit, dependency lock)
simulate <file.x3b> [--gas N]	Dry-run bytecode on the VM, print receipt/stat counts
run <file.x3b> [--gas N]	Like simulate but exits non-zero on failure
explain <file.x3b>	Disassemble bytecode into a readable per-instruction trace

<code>fmt <file> [--check]</code>	Pretty-print / format source
<code>lint <file></code>	Static-analysis linter
<code>estimate <file></code>	Estimate artifact cost (instruction counts, VM weight, payload bytes) — explicitly does not guess EVM gas, SVM compute, latency, or fees
<code>graph <file> --from A --to B [--max-hops] [--max-slippage-bps] [--min-liquidity]</code>	Build an opportunity graph from declared venues and search for routes
<code>optimize <file> --from A --to B [--objective]</code>	Deterministically choose one route from the opportunity graph
<code>fusion <file></code>	Find rings of intents that could settle against each other instead of using external liquidity
<code>netting <file></code>	Offset a book of obligations; report the residual that must actually move
<code>score <file></code>	Compute route/ risk score for an intent
<code>test <file> [--generate-only]</code>	Generate and run tests for an intent
<code>fuzz <file> [--iterations N]</code>	

	Generate fuzz tests
<code>chaos <file> [--scenarios N]</code>	Generate chaos-test scenarios
<code>audit <file> [--mode mainnet]</code>	Run a mainnet safety audit on an intent
<code>plan <file> [--show-route] [--json]</code>	Show the execution plan for an intent
<code>inspect <file> [--json]</code>	Inspect compiled intent/bytecode
<code>deploy <file> [--target] [--mode]</code>	Compile and print a deployment plan
<code>refund <intent_hash> [--check-only]</code>	Inspect or trigger a refund for an intent
<code>new <name> [--path]</code>	Scaffold a new X3 project
<code>receipt inspect <receipt.json></code>	Print a receipt as canonical JSON
<code>receipt verify <receipt.json></code>	Verify a receipt's hash and accounting invariants
<code>receipt execute <file> [--block N] [--key-hex H]</code>	Compile a trading program, execute it against a neutral fixture host, and emit a signed receipt
<code>packet inspect <input></code>	Print a solver's opportunity packet and its committed hashes
<code>packet verify <input> --trusted KEY=HEX</code>	Verify a packet's structure, commitments, deadline, and signature against

	named trusted keys
--	-----------------------

WHAT "RECEIPT EXECUTE" ACTUALLY PROVES

The tool's own documentation is explicit: `x3c receipt execute` proves the compile → execute → receipt → sign → verify pipeline connects end to end against a fixture host where every swap simply returns the trade's declared `min_output` and fees are zero. It does not simulate real market profitability. A program whose guards genuinely depend on real price movement can still legitimately fail against a live venue even though it passes here.

CHAPTER 9

Proof by Test

x3-lang's atomicity claim is the centerpiece of this book, so it deserves the strongest evidence available: not documentation, but tests that were actually run while writing this book.

9.1 Rollback across a real compile → VM → bridge-adapter path

`vm/tests/test_e2e_examples.rs::e2e_atomic_rollback_on_bridge_failure` compiles a real example file (`examples/timeout_refund_minimal.x3`) through the full `compile_source` pipeline, wires a deliberately failing bridge backend (`FailingBackend`) that errors on `verify_source_finality` — and asserts via `unreachable!()` that `verify_transfer_proof` and `persist_receipt` are never reached on the failure path — then runs the compiled bytecode on the real VM through a `ProductionBridgeAdapter` and checks that the atomic-rollback path fires correctly. All 12 tests in this file pass, including the happy-path sibling `e2e_bridge_source_executes_through_production_adapter`, which asserts finality is checked exactly once, a proof is verified exactly once, and exactly one receipt is persisted with the correct amount and a `x3-settlement-receipt:v1:`-prefixed encoding.

9.2 Rollback as a property, not a single example

`vm/tests/property_tests.rs::prop_atomic_rollback_restores_registers` uses `proptest` to generate arbitrary register pairs `(a, b)`, runs them through `ATOMIC_BEGIN → ADD → ATOMIC_ROLLBACK → REQUIRE → HALT`, and asserts that for *every* generated input the registers end up exactly back at their pre-`ATOMIC_BEGIN` values. A sibling test, `prop_rollback_without_begin_always_fails`, asserts that an `ATOMIC_ROLLBACK` with no matching `ATOMIC_BEGIN` deterministically errors rather than silently doing nothing.

9.3 The full Trading Core pipeline, end to end

`compiler/tests/test_trading_core_e2e.rs` takes `examples/trading_core_v1.x3` itself (via `include_str!`, so the test and the book's Chapter 5.6 example are guaranteed to be the same file) and runs it through `parse_source → analyze_trading / verify_trading_program → lower_atomic_trade → execution` against a `FixtureVenueHost` implementing the real `TradingHost` trait → `build_receipt → sign_receipt → verify_receipt_trusted`. This is exactly the pipeline exposed to end users through `x3c receipt execute`.

SUMMARY

Atomic rollback is proven both as a concrete end-to-end scenario (9.1) and as a property that holds across randomized inputs (9.2). The compile-to-signed-receipt pipeline for Trading Core is proven end to end against the same example this book teaches from (9.3). These are the strongest, most defensible technical claims in the whole project.

CHAPTER 10

Current Status & Honest Limitations

Per this project's own source-of-truth rule — trust code over roadmap prose — this chapter exists so the book doesn't overclaim what x3-lang does today.

10.1 What the README states outright

Trading Core v1's explicit v1 boundaries, verbatim:

- No live Aave, Uniswap, SushiSwap, bridge, or private-relay adapters.
- No route discovery or oracle aggregation.
- Fixture capabilities are rejected in production/mainnet execution mode.
- No live-DEX or mainnet claim is made without real adapter evidence.

The atomic-router feature's tracked readiness score is 85%, described as reflecting "remaining production hardening: multi-validator atomic execution tests, external bridge integration, and CI gate wiring — not missing VM functionality." In other words: the opcodes genuinely execute; what's incomplete is breadth of integration testing, not the execution model itself.

10.2 What actually compiles — an honest sweep

Every file in `x3-lang/examples/` was run through `x3c check` while researching this book. Seventeen of twenty-two compile cleanly; five do not:

File	Result
simple_swap.x3, atomic_swap.x3, timeout_refund.x3, timeout_refund_minimal.x3, staking_intent.x3, route_fallback.x3, trading_core_v1.x3, trading_effects.x3, atomic_choice.x3, flagship_b52.x3, intent_fusion.x3, mainnet_safe_swap.x3, multi_leg_route.x3, objective_routing.x3, opportunity_graph.x3, parallel_dag.x3, strategy_module.x3	COMPILES
arb.x3, flash.x3, jit_lp.x3, mev_smooth.x3, x3_coin_layer.x3	FAILS TO PARSE — Solidity-flavored pseudocode (<code>contract {}</code> , <code>u256</code> , <code>@decorators</code>) left over from an earlier design iteration; not valid x3-lang
arb_solana_eth.x3	SEMANTIC ERROR — the README's own "quick start" example uses <code>require</code>

```
proof verified, a guard kind the Rust
compiler doesn't recognize (only
proof_complete is); written in the
Python DSL's vocabulary, not the Rust
DSL's
```

Practical takeaway: if you find a new `.x3` file in this repository and want to know whether it represents working language behavior, run it through `x3c check` before trusting it — do not assume every file in `examples/` is current.

10.3 The two "capability catalog" spec documents

`spec/x3-unique-capabilities.md` (roughly 50 catalogued capabilities — GPU swarm scheduling, ZK/MPC proof verification, CRDT state, VRF randomness, AI-agent sandboxing, and more) and `spec/x3-profit-primitives.md` (roughly 50 catalogued trading strategies — MEV protection, liquidation bots, cross-chain yield aggregation, and more) are vision/roadmap documents. Their code samples use a Python-like colon-block syntax (`fn foo(a: type) -> type:`, indentation-based blocks, `@decorator` annotations) that does not match the real, implemented grammar at all — there is no colon-terminated function header anywhere in the actual lexer or parser. Treat every code sample in those two files as illustrative pseudocode describing a direction, not as a description of shipped behavior.

10.4 What's genuinely solid

To close on the right note: the parse → semantic-check → lower-to-IR → validate-IR → emit-bytecode → verify-bytecode → execute → commit-or-rollback → settle → verify-receipt pipeline described in Chapters 6–7 is real, wired, and covered by passing tests (Chapter 9) — including property-based tests that hold across randomized inputs, not just hand-picked fixtures. Seventeen real example programs compile and run today. The gap between "vision documents" and "shipped language" is wide in this project, but the shipped core — atomic cross-VM intents with VM-enforced rollback and signed receipts — is real, not vaporware.

APPENDIX A

Full Opcode Table

Merged from `spec/opcodes.rs` (authoritative — compiled directly into the compiler and VM crates via `include!`) and `spec/opcodes.yaml` (a reference file that is missing several newer opcodes, marked below).

Opcode	Hex	Category	Description
NOP	0x00	Misc	No operation
ADD	0x01	Arithmetic	Register add
SUB	0x02	Arithmetic	Register subtract
META_NONCE	0x10	Metadata	Transaction nonce metadata
META_CHAIN_ID	0x11	Metadata	Chain identifier metadata
LOCK	0x20	Asset ops	Lock assets on source chain
MINT	0x21	Asset ops	Mint assets on destination chain
BURN	0x22	Asset ops	Burn assets on source chain
RELEASE	0x23	Asset ops	Release locked assets
SWAP	0x24	Asset ops	Atomic swap between assets
BRIDGE	0x25	Bridge	Cross-chain bridge operation
IF	0x30	Control flow	Conditional branch
LOOP	0x31	Control flow	Bounded loop construct
CALL	0x32	Control flow	Call subroutine
RET	0x33	Control flow	Return from subroutine
REQUIRE	0x40	Guards	Assert precondition; traps to nearest ON_FAIL on failure
ON_FAIL	0x41	Guards	Register a failure handler
ON_TIMEOUT	0x42	Guards	Register a timeout / deadline handler
ATOMIC_BEGIN	0x50	Atomic	Snapshot VM state
ATOMIC_END	0x51	Atomic	Commit snapshot
ATOMIC_ROLLBACK	0x52	Atomic	Restore snapshot exactly

ATOMIC_CHOICE (not in .yaml)	0x53	Atomic	Record which of several verified branches was selected
ROUTE_FALLBACK (not in .yaml)	0x54	Atomic	Record an approved, bounded route substitution
PARALLEL_PLAN (not in .yaml)	0x55	Atomic	Record a parallel execution plan
FEATURE_ALLOW (not in .yaml)	0x56	Atomic	Record which execution-mode features a program consents to
STRATEGY_LICENSE (not in .yaml)	0x57	Atomic	Strategy-module licensing metadata
EMIT	0x60	External calls	Emit event / EVM-side call via bridge adapter
CALL_HOST	0x61	External calls	Call host function / SVM-side call via bridge adapter
RESERVED_70...RESERVED_7B	0x70-0x7B	Reserved	Unused placeholders
GPU_DISPATCH	0x80	Capability	Dispatch GPU compute kernel
SIMULATE	0x81	Capability	Simulate execution
SCHEDULED_DISPATCH	0x82	Capability	Scheduled dispatch operation
INTENT_RESOLVE	0x83	Capability	Resolve user intent
CRDT_OP	0x84	Capability	CRDT operation
PROOF_VERIFY	0x85	Capability	Verify cryptographic proof
STORAGE_OP	0x86	Capability	Storage operation
PATHFIND	0x87	Capability	Pathfinding operation
MEMPOOL_SCAN	0x88	Capability	Mempool scan operation
ORACLE_REQUEST	0x89	Capability	Oracle data request
EMERGENCY_CONTROL	0x8A	Capability	Emergency control operation
LIFECYCLE	0x8B	Capability	Lifecycle management
SERIALIZE	0x8C	Capability	Serialize data
DESERIALIZE	0x8D	Capability	Deserialize data
GAS_ESTIMATE	0x8E	Capability	Estimate gas cost
CHAIN_METRIC	0x8F	Capability	Read chain metric
EVENT_PROVENANCE	0x90	Capability	Event provenance tracking
MULTI_HOP_SWAP	0x91	Capability	Multi-hop token swap
VECTOR_MATH	0x92	Capability	Vector math operation

ROLE_CHECK	0x93	Capability	Check role permission
MULTISIG_CHECK	0x94	Capability	Multisig verification
VERSION_META	0x95	Capability	Version metadata
STORAGE_NAMESPACE	0x96	Capability	Storage namespace operation
ABI_EXPORT	0x97	Capability	Export ABI interface
DOC_EMBED	0x98	Capability	Embed documentation
GAS_ADAPTIVE	0x99	Capability	Adaptive gas pricing
BOUNTY	0x9A	Capability	Bounty operation
SUB_EXEC	0x9B	Capability	Sub-execution context
NONCE_UNUSED (not in .yaml)	0x9C	Capability	Check and record nonce freshness
ROUTE_SCORE	0xA0	B-52	Route scoring operation
SOLVER_BID	0xA1	B-52	Solver bid submission
RELAYER_ATTEST	0xA2	B-52	Relayer attestation
RPC_CONSENSUS	0xA3	B-52	RPC consensus check
RISK_SCORE	0xA4	B-52	Risk score evaluation
INVARIANT_CHECK	0xA5	B-52	Invariant check
PRIVACY_COMMIT	0xA6	B-52	Privacy commitment
PROOF_REQUIRED	0xA7	B-52	Proof-required check
VM_ADAPTER_CALL	0xA8	B-52	VM adapter call
MODE_CHECK	0xA9	B-52	Compilation mode check
PACKAGE_IMPORT	0xAA	B-52	Package import
REFUND_POLICY	0xAB	B-52	Refund policy check
TRADING_BEGIN	0xB0	Trading core	Begin an atomic trading region
TRADING_OPEN_DEBT	0xB1	Trading core	Open a linear trading debt (borrow)
TRADING_EXECUTE_SWAP	0xB2	Trading core	Execute an explicit venue swap
TRADING_CLOSE_DEBT	0xB3	Trading core	Close a trading debt (repay)
TRADING_ASSERT_MIN_PROFIT	0xB4	Trading core	Assert minimum net profit

TRADING_ASSERT_ALL_DEBTS	0xB5	Trading core	Assert all debts are closed
TRADING_EMIT_RECEIPT	0xB6	Trading core	Emit deterministic signed receipt
TRADING_COMMIT	0xB7	Trading core	Commit the atomic trade
TRADING_ABORT	0xB8	Trading core	Abort the atomic trade
TRADING_ASSERT_INVARIANT (not in .yaml)	0xB9	Trading core	Assert a named invariant
TRADING_BRIDGE (not in .yaml)	0xBA	Trading core	Cross-chain leg within a trading program
HALT	0xFF	Misc	Halt execution

APPENDIX B

Quick Reference Card

Compile and run a program

```
cargo run -p x3-tools --bin x3c -- check examples/simple_swap.x3
cargo run -p x3-tools --bin x3c -- build examples/simple_swap.x3 -o out.x3b
cargo run -p x3-tools --bin x3c -- run out.x3b
cargo run -p x3-tools --bin x3c -- explain out.x3b
```

Trading Core: compile, execute, and verify a receipt

```
cargo run -p x3-tools --bin x3c -- receipt execute examples/trading_core_v1.x3 -o
receipt.json
cargo run -p x3-tools --bin x3c -- receipt verify receipt.json
```

Intent skeleton to copy

```
intent <name> {
  from <chain>.<ASSET> amount <N>
  to <chain>.<ASSET> receiver <addr>
  route { bridge <bridge> <chain>.<ASSET> -> <chain>.<ASSET> receiver <addr> }
  require nonce unused <id>
  require finality.<chain> >= <N>
  timeout <duration> refund <chain>.<ASSET> to sender
  on_fail rollback
}
```

Atomic trade skeleton to copy

```
atomic trade <Name> using <Policy> {
  borrow <N> <ASSET> from <venue> as debt
  let x = swap debt.amount <ASSET> -> <ASSET> via <venue> min_out <N> <ASSET>
  repay debt
  require net_profit >= <N> <ASSET>
  require all_debts_repaid
}
```

```
    emit receipt  
}
```

Before you trust a new .x3 file

```
cargo run -p x3-tools --bin x3c -- check <file.x3>  
# "no semantic errors" → real, compiling x3-lang  
# "expected top-level item" → stale/pseudocode, not the current grammar
```

This book was compiled from the x3-lang workspace source ([x3-lang/](#)), the language grammar spec ([docs/openspec/x3-language-grammar.md](#)), the opcode specification ([x3-lang/spec/opcodes.rs](#)), and passing test suites in [compiler/tests/](#) and [vm/tests/](#) . Every code sample was compiled against the real [x3c](#) toolchain rather than transcribed from documentation.